

**Reading Egyptian Hieroglyphs Automatically:
A Technical Report on the Design, Machine Learning
Methods, and
Engineering Issues of a Photo-to-Gloss Pipeline**

Delfin Eryilmaz

September 15, 2026

Abstract

This report explains, from first principles, how a piece of software takes a photograph of Egyptian hieroglyphs and produces, for every sign in the photo, its Gardiner sign-list code, its transliteration (when one is known), and an English gloss. The report assumes no prior knowledge of machine learning: every technical term (neural network, convolution, transfer learning, object detection, fine-tuning) is defined the first time it is used, and each definition is followed by a concrete example drawn from this project. The system is built from three stages that run one after another: finding where each glyph sits in the photograph (segmentation), deciding which of 171 known signs each glyph is (classification), and assembling the results into a readable output (reading-order sorting and dictionary lookup). Two implementations of the first stage are covered in detail: a classical image-processing approach that needs no training data, and a newer approach that fine-tunes a pretrained neural network on synthetically generated training images. The report also documents a real defect that was found and fixed during the development of the second approach, as a worked example of how a plausible-looking machine learning pipeline can fail for a subtle reason that only becomes visible when it is actually run.

Contents

1	Introduction	2
1.1	What this project does	2
1.2	Why this is a machine learning problem	2
1.3	Report roadmap	3
2	Background: How a Computer “Sees” and “Learns”	4
2.1	Images as numbers	4
2.2	What machine learning means here	4
2.3	Neural networks, from scratch	5
2.4	Convolutional neural networks	6
3	Finding Each Glyph in a Photograph (Segmentation)	8
3.1	The task: from photograph to bounding boxes	8
3.2	A classical computer vision approach	8
3.3	Object detection versus image classification	9
3.4	Building a trained detector	10
3.4.1	The missing-data problem	10
3.4.2	Synthesizing training data	11
3.4.3	Preventing data leakage	12
3.4.4	Fine-tuning from a pretrained checkpoint	12
3.4.5	A real technical issue: mismatched label formats	13
3.4.6	Evaluation plan	14
4	Identifying Each Sign (Classification)	15
4.1	The task	15
4.2	The dataset	15
4.3	Transfer learning: reusing a pretrained model	15
4.4	Training details	17
4.5	Results	17
5	Putting It Together: Ordering and Looking Up Signs	18
5.1	Reading order	18
5.2	Looking up each sign’s meaning	18
5.3	The complete pipeline	20
6	The Demo Application	21
7	Known Limitations and Future Work	22
8	Summary	23

Chapter 1

Introduction

1.1 What this project does

The input to this system is an ordinary photograph containing one or more Egyptian hieroglyphic signs, for example a photograph of a carved wall, a papyrus, or a single inscribed object. The output is a list, one entry per sign found in the photograph, each entry giving:

- (1) the sign’s **Gardiner code**, a short alphanumeric label such as **D21** or **G17** that Egyptologists use to refer to a specific hieroglyphic sign, following the classification scheme published by Sir Alan Gardiner in 1927 and still in standard use today;
- (2) the sign’s **transliteration**, a short sequence of Latin letters approximating how the sign was pronounced or the sound value it represents, when one is recorded for that sign (many signs, called determinatives, carry meaning but no sound and genuinely have no transliteration);
- (3) the sign’s **English gloss**, a short English word or phrase describing what the sign means.

It is important to state plainly what the system does *not* do. It does not produce a fluent English sentence translation of an inscription. Translating hieroglyphic text into readable English requires knowledge of Middle Egyptian grammar (word order, verb conjugation, determinative conventions) that a per-sign lookup table cannot supply, and no large corpus of hieroglyphic text paired with human-translated English sentences exists to train such a system on. The output is honestly presented as what it is: a list of individual sign meanings in the order the signs were read, not a sentence.

1.2 Why this is a machine learning problem

A photograph of a hieroglyph is, to a computer, nothing more than a grid of numbers: each pixel is stored as one or more numbers describing its brightness and color. Section 2.1 explains this representation in detail. The task of turning that grid of numbers into “this is sign D21, the mouth glyph” cannot be solved by writing an explicit rule such as “if the pixels form this exact shape, it is D21”, because photographs of the same sign vary enormously: different carvings, different lighting, different camera angles, and different amounts of weathering all change the exact pixel values while the sign’s identity stays the same. Machine learning is the family of techniques used when the mapping from input to output is too complex and too variable to write down as an explicit rule by hand, and is instead learned automatically from many labeled examples. Section 2.2 defines this more precisely.

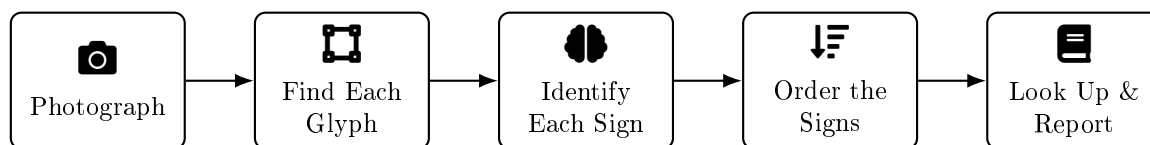


Figure 1.1: The five stages this report walks through, from an input photograph to the final list of Gardiner codes, transliterations, and glosses. Each stage is a separate chapter or section below.

1.3 Report roadmap

Figure 1.1 shows the shape of this report at a glance. Chapter 2 builds up the machine learning vocabulary used throughout the rest of the report, starting from what an image looks like to a computer and ending at convolutional neural networks, the specific kind of neural network this project uses. Chapter 3 covers the first pipeline stage, finding where the glyphs are in a photograph, including both the classical (non-learned) approach already in production and a newer approach, still being trained at the time of writing, that fine-tunes a neural object detector. Chapter 4 covers the second stage, identifying which of the 171 known signs each found glyph is. Chapter 5 covers how the pipeline puts the per-glyph results back together into an ordered, human-readable list. Chapter 6 describes the web application built on top of the pipeline. Chapter 7 states, without softening, what the system does not yet do well and why.

Chapter 2

Background: How a Computer “Sees” and “Learns”

This chapter defines every piece of machine learning vocabulary used elsewhere in the report. A reader already familiar with neural networks and convolutional neural networks can skip to Chapter 3 without losing anything needed later.

2.1 Images as numbers

A digital photograph is stored as a grid of small squares called pixels. Each pixel holds one number (for a grayscale image, where the pixel is some shade between black and white) or three numbers (for a color image, one each for red, green, and blue light intensity). A typical grayscale pixel value runs from 0 (pure black) to 255 (pure white); a mid-gray pixel might be 128. A small grayscale image, say 50 pixels wide by 75 pixels tall, is therefore nothing more than a table of $50 \times 75 = 3,750$ numbers between 0 and 255. This project’s single-glyph training images are exactly that size and format: 50 by 75 pixel grayscale crops, each pixel a single brightness number.

This matters because every operation described in this report, whether it is a classical image-processing step or a neural network, is ultimately doing arithmetic on these grids of numbers. There is no step at which the computer “looks at” the image the way a person does; every stage described below reduces to additions, multiplications, comparisons, and lookups performed on a table of numbers.

2.2 What machine learning means here

Machine learning, in the specific sense used throughout this report, means building a function that maps an input (here, a grid of pixel numbers) to an output (here, a sign identity, or a set of bounding boxes), where the exact arithmetic that function performs is not written by a programmer but is instead adjusted automatically based on many examples of correct input-output pairs. This process of automatic adjustment is called **training**. A concrete example used throughout this report: the sign classifier described in Chapter 4 is trained on 4,031 single-glyph images, each one labeled with which of 171 Gardiner codes it is; training adjusts the classifier’s internal arithmetic so that, given a new image it has never seen, it outputs the correct code as often as possible.

Three terms recur throughout this report and are worth fixing precisely before they appear again.

- **Training data** is the collection of labeled examples used to adjust the model. For the classifier, this is the 4,031 labeled glyph crops (Section 4.2). For the object detector described in Chapter 3, no labeled real-world photographs exist, so the training data is synthetically generated (Section 3.4.2).
- **A model** is the function being trained, together with the specific numeric values (called **weights**) it currently holds. Two models with the same structure but different weights, for example before and after training, behave completely differently.
- **Inference** is the act of running an already-trained model on a new input to get an output, as opposed to training, which is the act of adjusting the model’s weights. Once a model is trained, running it on the Streamlit demo application (Chapter 6) is inference, not training; no further learning happens at that point.

2.3 Neural networks, from scratch

A **neural network** is one specific kind of trainable function, loosely inspired by connections between neurons in a brain, though the analogy should not be taken literally: the arithmetic involved is ordinary linear algebra, not a simulation of biological neurons.

A neural network is organized into **layers**. Each layer is a set of units, historically called neurons, arranged side by side. The first layer, the **input layer**, holds the raw input numbers (for an image, the pixel values, laid out in some order). The last layer, the **output layer**, holds the network’s answer (for a classifier choosing among 171 signs, the output layer has 171 units, one per possible sign). Any layers between the input and output are called **hidden layers**.

Every unit in one layer is connected to every unit in the next layer by a **weight**, a single adjustable number. A unit’s value is computed as a weighted sum of every unit’s value in the previous layer (each multiplied by its connecting weight and added together), passed through a simple nonlinear function called an **activation function** that decides how strongly the unit “fires”. Figure 2.1 shows this structure for a small example network. **Training** a neural network means searching for the specific set of weight values that makes the network’s output layer agree with the correct label as often as possible across the training data, using an algorithm called backpropagation combined with gradient descent; this project does not implement that search algorithm itself, since it is provided by the PyTorch library, but the report returns to what is actually being adjusted (Section 4.3) when it discusses how the sign classifier is trained.

Worked example

Suppose, for illustration, a tiny network is being trained to tell apart two signs using only 4 input numbers (a drastically simplified stand-in for pixel values) and the 2-unit output layer shown in Figure 2.1. Before training, the weights are set to small random numbers, so the network’s two output units produce essentially random scores for any input. Training repeatedly shows the network a labeled example (an input together with which of the two signs it actually is), compares the network’s output to the correct answer, and nudges every weight in the network slightly in the direction that would have made the output closer to correct for that example. Repeated over thousands of examples, this nudging process converges on a set of weights where the correct output unit tends to score highest for the corresponding input. The single arrow labeled “one weight” in Figure 2.1 is one such adjustable number; a real network in this project has hundreds of thousands of them, but the training principle (compare to a labeled example, nudge every weight slightly) is unchanged.

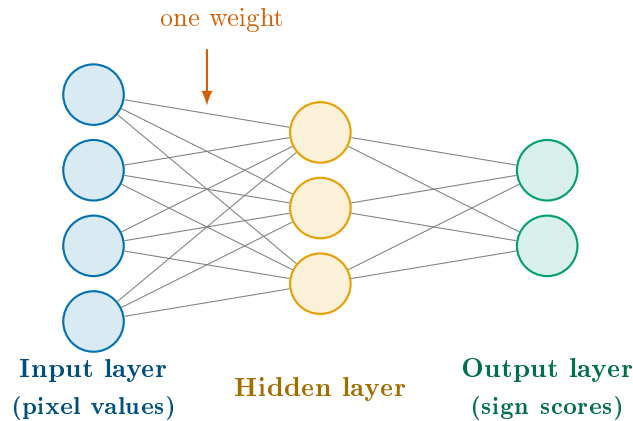


Figure 2.1: A small neural network with one input layer (4 units), one hidden layer (3 units), and one output layer (2 units). Every line is a weight, a number adjusted during training. A real classifier layer in this project has far more units per layer than shown here; the structure is the same, only the size differs.

2.4 Convolutional neural networks

The network described in Section 2.3 connects every input unit to every hidden unit. Applied directly to an image, this becomes impractical almost immediately: a modest 224 by 224 pixel color image has $224 \times 224 \times 3 = 150,528$ input numbers, and a hidden layer of a useful size would need tens of millions of weights just for the first layer, most of which would be redundant, since a pattern that identifies a curved stroke in the top-left corner of an image is, for the most part, the exact same pattern that would identify a curved stroke anywhere else in the image.

A **convolutional neural network**, or **CNN**, solves this by reusing the same small set of weights at every position in the image, instead of learning a separate weight for every pixel-to-unit connection. A small grid of weights, called a **filter** or **kernel** (typically as small as 3×3 pixels), is slid across the entire input image. At every position, the filter's weights are multiplied against the pixels currently underneath it and summed into a single number; sliding the filter across the whole image produces a grid of these summed numbers called a **feature map**. Because the same filter is reused everywhere, a filter that has learned to respond strongly to (for example) a horizontal edge will detect horizontal edges anywhere in the image, using a tiny fraction of the weights a fully-connected layer would need. A convolutional layer normally learns many filters at once (tens to hundreds), each producing its own feature map, so that different filters can specialize in detecting different simple patterns (edges at different angles, corners, small textures).

CNNs stack many convolutional layers on top of each other. Layers close to the input tend to learn simple, general-purpose patterns such as edges and corners; feature maps from an early layer feed into a later convolutional layer, whose filters combine those simple patterns into more complex ones (for example, a curve plus a straight edge might combine into something resembling part of a bird's beak); layers near the end of the network combine complex patterns into whole-object concepts. Between convolutional layers, a **pooling** step commonly shrinks each feature map (for example, by keeping only the largest value in each small block of the map), which reduces the amount of computation needed by later layers and makes the network somewhat tolerant to small shifts in where a pattern appears. Figure 2.2 shows this structure end to end, from an input image to a final classification.

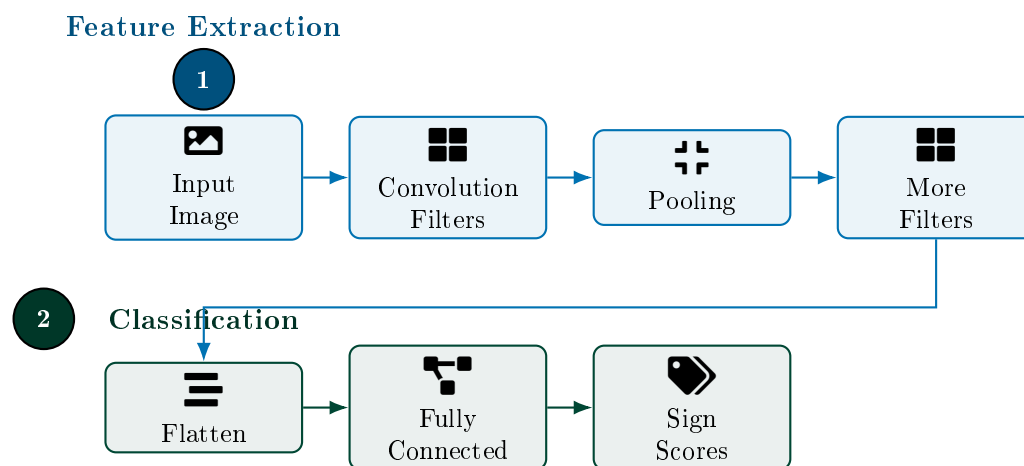


Figure 2.2: A convolutional neural network, in two phases. Feature extraction (blue) repeatedly applies small filters and pooling to build up increasingly complex patterns from the raw pixels. Classification (green) flattens the final feature maps into a single list of numbers and passes them through a fully connected layer (the kind described in Section 2.3) to produce one score per possible sign. The sign classifier in this project, described in Section 4.3, is a CNN of this shape called ResNet18.

Worked example

Consider a single 3×3 edge-detecting filter with weights arranged so that it produces a large output when the pixels underneath it go from dark on the left to light on the right, and a small output otherwise. Sliding this filter across a photograph of the sign **D21** (a mouth, drawn as a horizontal curved shape) produces a feature map that lights up strongly along the sign’s left-facing edges and stays low elsewhere. That feature map is itself just another grid of numbers, so a second convolutional layer can be applied to it exactly as the first layer was applied to the original image, this time detecting combinations of edges rather than raw pixel contrast. Repeating this a handful of times, exactly as shown in Figure 2.2, is how a CNN builds up from “this pixel is darker than its neighbor” to “this arrangement of edges and curves looks like a **D21**”.

Chapter 3

Finding Each Glyph in a Photograph (Segmentation)

3.1 The task: from photograph to bounding boxes

Before any sign can be identified, the system first has to answer a simpler question: where in the photograph does each individual sign sit? This is called **segmentation** in this project (the more general term in the wider computer vision field is **object detection**, discussed in Section 3.3). The output of this stage is a list of **bounding boxes**, one per glyph found in the photograph. A bounding box is the smallest upright rectangle that contains one glyph, stored as four numbers: an x and y position for its top-left corner, and a width and height. Every later stage of the pipeline (classification, reading-order sorting, lookup) operates on the region of the photograph inside one bounding box at a time, never on the whole photograph at once.

This project implements two different ways of producing these bounding boxes, sharing one common interface (an abstract base class named **Segmenter** in the source code) so that either one can be plugged into the rest of the pipeline without changing anything downstream. The first, described in Section 3.2, is a classical image-processing method that needs no training data and is the one currently used by default. The second, described in Section 3.4, is a neural network trained specifically to find hieroglyphs, and is expected to perform better on difficult, real-world photographs once training completes.

3.2 A classical computer vision approach

The classical segmenter finds glyphs using a fixed sequence of image processing steps, none of which involve a trained model or any adjustable weights, only local pixel arithmetic and long-established computer vision techniques. Figure 3.1 shows the five steps in order.

- (1) **Convert to grayscale.** If the input photograph has separate red, green, and blue channels, they are combined into a single brightness channel, since the remaining steps only need brightness, not color.
- (2) **Blur to reduce noise.** A small Gaussian blur (averaging each pixel with its immediate neighbors) is applied first, so that isolated single-pixel speckles of noise do not get mistaken for real detail by the next step.
- (3) **Binarize (threshold).** Every pixel is converted to either pure black or pure white, based on whether it is darker or lighter than a cutoff brightness. The implementation defaults

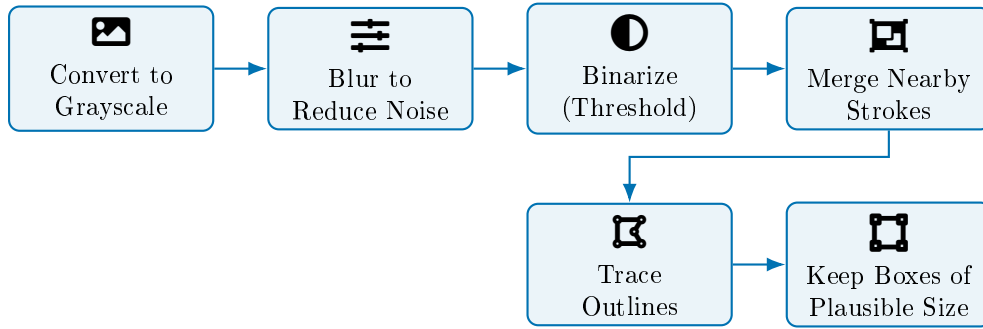


Figure 3.1: The classical segmenter’s five processing steps, applied to every input photograph in order. None of these steps involve a trained model; every step is a fixed image-processing rule.

to **adaptive** thresholding, meaning the cutoff is computed separately for each local neighborhood of the image (a 25-pixel-wide window, in the current configuration) rather than using one single cutoff for the whole photo; this copes better with a photograph that is more brightly lit on one side than the other, since a single global cutoff would misclassify an evenly-lit dark carving as “all background” the moment part of the photo gets brighter than the cutoff. The method assumes glyphs are darker than their surrounding surface, which holds for incised or shadowed carvings, the expected common case.

- (4) **Merge nearby strokes.** A single hieroglyphic sign is frequently drawn as several separate pen or chisel strokes rather than one connected shape. A morphological operation called **closing** (a controlled expansion of the white regions, followed by an equal controlled shrink) is applied so that strokes belonging to the same glyph, which are usually close together, get merged into one connected white blob, while strokes belonging to different, more distant glyphs stay separate.
- (5) **Trace outlines and keep plausible boxes.** Every connected white blob’s outline (its **contour**) is traced, and the smallest upright rectangle containing that outline becomes a candidate bounding box. Candidates that are extremely small (likely leftover noise) or extremely large (likely a sign that the whole thresholding step failed on, so the “blob” is actually most of the photograph) are discarded; the remaining boxes are the segmenter’s final output.

Why this approach struggles on real photographs

This method’s central weakness follows directly from step 3. Adaptive thresholding works well when there is a reasonably consistent brightness difference between a glyph and its background across the whole photograph. On a clean, evenly-lit photograph of an isolated inscription, this assumption usually holds. On a heavily weathered wall, a photograph with strong shadows falling unevenly across the surface, or a cluttered scene with other objects in frame, the assumption breaks down: the same thresholding cutoff that correctly separates a glyph from background in one part of the image can incorrectly merge glyph and background, or split one glyph into fragments, in another part of the same image. Because this approach has no way to learn from examples what a hieroglyph actually looks like, it cannot recover from this failure the way a trained model potentially could; it can only follow its fixed rule.

3.3 Object detection versus image classification

Before describing the trained detector, it is worth making one distinction precise, since it explains why segmentation and the identification stage in Chapter 4 are built as two separate

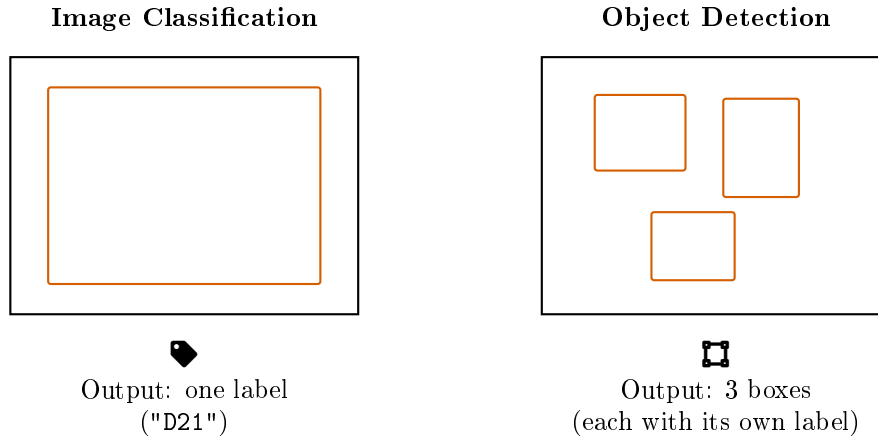


Figure 3.2: Image classification produces one label for a whole image. Object detection produces a variable number of bounding boxes, each labeled separately. Segmentation (Section 3.2 and Section 3.4) is this project’s detection stage; the per-glyph classifier (Chapter 4) is a classification stage that runs once per detected box, not once per photograph.

systems rather than one. **Image classification** answers the question “what is the single main thing in this image?” and produces one label for the whole image. **Object detection** answers the harder question “where are all the things of interest in this image, and what is each one?” and produces a list of bounding boxes, each with its own label. Figure 3.2 contrasts the two.

This project’s classifier (Chapter 4) is a classification model: given one cropped image known in advance to contain exactly one glyph, it outputs one Gardiner code. It has no way to look at a full photograph and say how many glyphs are in it or where they are; that is exactly the job segmentation does first. The two trained-detector components described in the rest of this chapter (the object detector itself, Section 3.4) locate glyphs without identifying them (the detector’s training data, described in Section 3.4.2, uses a single generic label, “hieroglyph”, for every box); identifying which specific sign each box contains remains the classifier’s job, applied separately to every box the detector finds. Keeping these two jobs separate means the detector never needs to be retrained just because a new sign class is added to the classifier, and the classifier never needs to know anything about where in a photograph its input crop came from.

3.4 Building a trained detector

The classical approach in Section 3.2 was, from the start of the project, deliberately placed behind the shared **Segmenter** interface described in Section 3.3, specifically so that a trained detector could be substituted in later without changing any other part of the pipeline. This section describes that substitute: a neural network trained to output bounding boxes directly, rather than relying on a fixed image-processing rule.

3.4.1 The missing-data problem

Training an object detector normally requires photographs that have already been manually annotated with bounding boxes, so the network has correct answers to learn from. No such dataset exists for this project. The classifier’s training data (Section 4.2) consists of 4,031 images, but every one of them is already a single-glyph crop: a photograph showing one sign and nothing else, with no bounding box annotation at all, because the whole image already is the glyph. A survey carried out before this stage was built (documented in [reports/2026-09-15-segmentation-detector-sourcing.md](#) in the project repository) con-

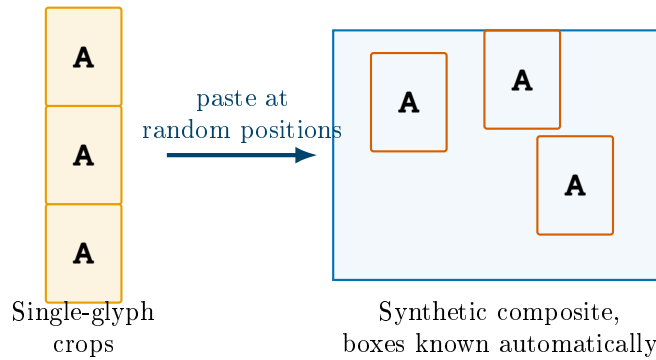


Figure 3.3: Composite image synthesis. Several single-glyph crops (left) are pasted onto a synthetic background canvas at randomly chosen, non-overlapping positions (right). Because the exact paste position and size of every crop is chosen by the program doing the pasting, the correct bounding box for every glyph in the resulting composite image is known with no manual annotation step.

firmed that no downloadable dataset of full, multi-glyph photographs with bounding-box annotations exists publicly either; the closest candidate found, a dataset of 235,436 hieroglyph instances referenced by a public GitHub repository, turned out not to actually distribute its images or labels, only a checkpoint already trained on them.

3.4.2 Synthesizing training data

Since no annotated multi-glyph photograph exists, this project manufactures its own detector training data from the single-glyph crops it already has, using a technique usually called **synthetic data generation** or **compositing**. The core idea: since every single-glyph crop’s exact extent is already known (it is simply the whole crop image), pasting one or more crops onto a blank canvas at known, randomly chosen positions produces a new, larger image in which the correct bounding box for each pasted glyph is known automatically, with no manual labeling step required at all. Figure 3.3 illustrates this process.

The implementation (the `synthesize` module in the project’s source code) refines this basic idea in three ways that matter for the resulting detector’s quality:

- (1) **Only the glyph’s ink is pasted, not its background.** Each single-glyph crop still has its own light background around the glyph (the crops are not pre-cut to the glyph’s silhouette). Pasting the crop verbatim would therefore stamp a visible rectangle of flat background color onto the canvas around every glyph, an artifact no real photograph has. Instead, only pixels darker than a fixed brightness cutoff (the same darker-than-background assumption used by the classical segmenter in Section 3.2) are copied onto the canvas; everything else is left as whatever was already on the canvas underneath.
- (2) **The background is not a single flat color.** An earlier version of this generator used one uniform gray value for the entire canvas. That was changed to add mild random brightness variation across the canvas (statistically, Gaussian noise around a base gray value), because a perfectly flat background never occurs in a real photograph, and Section 3.4.5 explains why training exclusively on flat backgrounds risked undermining the whole reason this detector is fine-tuned rather than trained from a blank state.
- (3) **Each pasted crop is randomly resized before placement.** A real photograph shows glyphs at many different apparent sizes, depending on how close the camera was and how large the sign was actually carved. Before being pasted, each crop is resized by a randomly chosen factor (currently between 70% and 130% of its original size), and the resulting bounding box reflects the resized dimensions, not the original crop’s dimensions.

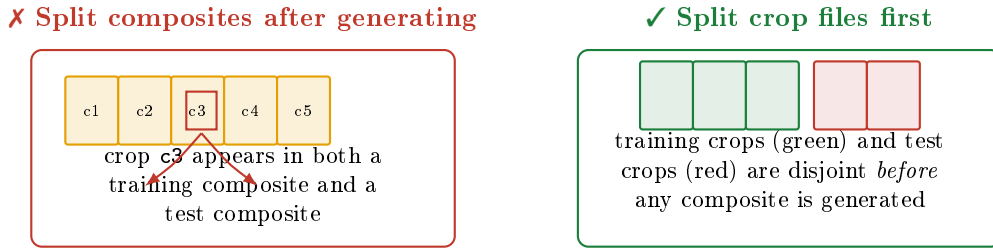


Figure 3.4: Preventing data leakage in the synthetic detector training set. Splitting composite images after they are generated (left) allows the same underlying crop to appear in both a training and a test composite. Splitting the pool of crop files first, and generating each split’s composites only from its own crops (right), guarantees no crop is ever shared between splits.

Crops are placed one at a time at random positions, rejecting a candidate position if it would overlap an already-placed crop by more than a small tolerance, and giving up on that particular crop (skipping it for this one composite) after a bounded number of failed attempts, so that a crowded canvas does not cause the generator to loop indefinitely trying to find room for one more glyph. The generator currently produces 4,000 composite images for training, 500 for validation, and 500 for a held-out test set (Section 3.4.3 explains how that split is chosen), each composite containing between 3 and 10 randomly chosen crops on a 640-by-640-pixel canvas.

3.4.3 Preventing data leakage

A trained model’s accuracy is normally measured on a **test set**: a portion of the labeled data deliberately held back from training and used only afterward, to check how well the model performs on examples it has never seen. **Data leakage** is the general name for any situation where information from the test set influences training, directly or indirectly, making the test-set accuracy number look better than the model’s real ability to handle genuinely new data. This project’s detector training has a specific, easy-to-miss leakage risk worth explaining explicitly, since it is a realistic mistake for exactly this kind of synthetic-data pipeline.

The obvious way to build a held-out test set here would be to generate one batch of composite images for training and a separate batch for testing. But if both batches are allowed to draw from the very same pool of single-glyph crop files, nothing stops the identical pixel data of one specific crop, say a particular photograph of a D21 sign, from appearing in a training composite and also, pasted at a different position, in a test composite. A detector could then score well on the test composite not because it has learned to recognize hieroglyphs in general, but because it has partially memorized that one exact photograph’s pixel pattern, an inflated and misleading accuracy number. Figure 3.4 contrasts the two approaches.

The fix implemented in this project splits the crop *files* into training, validation, and test pools (in an 80/10/10 proportion, using a fixed random shuffle so the split is reproducible) before any composite image is generated at all. Composite images for the test set are then built exclusively from the test pool’s crop files, and likewise for training and validation; no single crop file’s pixels can ever appear in more than one split’s composites, regardless of how many composite images are generated from each pool.

3.4.4 Fine-tuning from a pretrained checkpoint

Training an object detector completely from a random starting point typically needs a very large amount of training data to reach useful accuracy, far more than the roughly 4,000 synthetic composites described above could realistically provide on their own. This project instead uses **fine-tuning**: starting from a checkpoint (a saved set of weights) that has already been trained

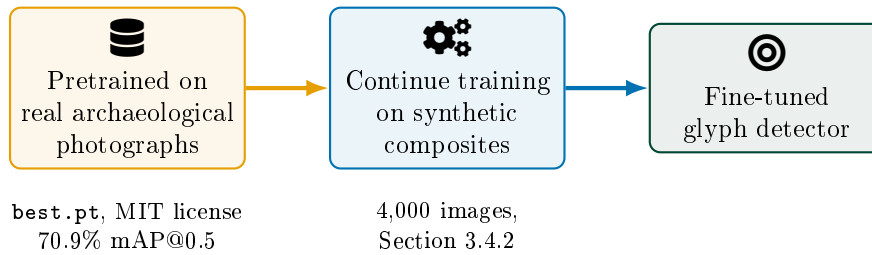


Figure 3.5: Fine-tuning the glyph detector. Training starts from a checkpoint already exposed to real archaeological photographs (orange), rather than from randomly initialized weights, then continues training on this project’s synthetic composite images (blue) to specialize the model for hieroglyph detection specifically.

by someone else on a different, larger dataset, and continuing training from there on this project’s own data for a smaller number of additional passes. Section 4.3 explains the closely related idea of transfer learning in more depth, in the context of the sign classifier; the same underlying principle applies here.

The specific checkpoint used is a YOLOv8 (“You Only Look Once”, a common family of real-time object detection architectures) segmentation model published under the MIT license by a separate, unaffiliated open-source project, already trained on real archaeological photographs of temple wall inscriptions, obelisk carvings, papyrus documents, and stelae, achieving a reported 70.9% mean average precision at an overlap threshold of 0.5 (a standard object-detection accuracy metric, abbreviated mAP@0.5, that need not be defined further here since this project does not itself claim that number, only starts from the checkpoint that earned it) on its own, separate evaluation photographs. Choosing this starting point over training from scratch is a deliberate bet: a model that has already learned what real, weathered stone carvings generally look like should adapt faster and generalize better to real photographs after a short additional training run than a model that has only ever seen this project’s flat synthetic composites. Figure 3.5 shows this process end to end.

3.4.5 A real technical issue: mismatched label formats

This section documents a genuine defect that was found and fixed during the development of this stage, kept in the report as a concrete example of how a machine learning pipeline can look correct at every individual step and still fail, for a reason that only becomes visible when the whole pipeline is actually run end to end.

Object detection models can be trained for (at least) two closely related but distinct tasks. A **detection** model is trained to output plain rectangular bounding boxes. A **segmentation** model (in the neural network sense, not to be confused with this project’s own use of “segmentation” for the whole find-the-glyphs stage) is trained to output a more precise outline, a polygon tracing the exact pixels belonging to each object, not just its enclosing rectangle. The pretrained checkpoint described in Section 3.4.4 is a segmentation model. Training-data label files therefore need to be written in the **segmentation label format**: each line names an object’s class, followed by the coordinates of a polygon outlining it. The code that originally generated this project’s training labels instead wrote the simpler **detection label format**: each line naming an object’s class, followed by only a center position, width, and height, five numbers in total rather than a polygon.

This mismatch produced no error anywhere in the label-writing code itself; five plausible-looking numbers were written to every label file, and every unit test that checked those numbers in isolation passed. The failure only appeared when the training library (Ultralytics, the library

Label line for one bounding box — before / after

```
- 0 0.878125 0.837500 0.125000 0.125000  
  (detection format:  class, center x, center y, width, height)  
  
+ 0 0.816 0.775 0.941 0.775 0.941 0.900 0.816 0.900  
  (segmentation format:  class, then the box's 4 corners as a polygon)
```

Figure 3.6: The label-format fix. The old line (red) gave the box’s center and size, the format a detection model’s loader expects. The new line (green) gives the same box’s four corners in order (top-left, top-right, bottom-right, bottom-left), the format the segmentation model’s loader actually requires. Both lines describe the exact same rectangle; only the encoding changed.

implementing YOLOv8) tried to actually load this training data for a segmentation model and found that every label was missing the polygon information a segmentation model’s loader requires, raising an error and refusing to proceed with training at all. Figure 3.6 shows the fix: a box’s own four corners, traced in order, are themselves a valid (if simple, rectangular) polygon outlining that box, so no additional information needs to be invented to satisfy the segmentation label format, only the same box’s coordinates need to be written out differently.

This defect was caught by a whole-branch code review carried out after every individual development task had already been reviewed and approved on its own. No single task’s review caught it, because the label-writing code, the checkpoint-downloading code, and the training notebook that combined the two were each written, tested, and reviewed as separate, independently reasonable pieces of work; the mismatch only existed in how two already-reviewed pieces fit together. The fix was verified not only by unit tests on the label-writing function in isolation, but by constructing a real Ultralytics segmentation dataset loader from generated training data and confirming it accepted the new format without error, the same check that had previously failed against the old format.

3.4.6 Evaluation plan

At the time of writing, the detector described in this section has not yet been trained; that training run, and the evaluation described here, are the next step in the project, intended to run on a cloud GPU service since no suitable GPU is available locally. Two separate checks are planned once training completes. The first is quantitative: mean average precision, computed on the held-out synthetic test composites described in Section 3.4.3, giving a single number comparable across different training runs. The second is qualitative: running both the classical segmenter (Section 3.2) and the newly trained detector on a small set of genuinely real photographs (a publicly available 40-image dataset, licensed for reuse, separate from both the training composites and the pretrained checkpoint’s own original training data) and comparing how many glyphs each one finds. This second check exists specifically because the quantitative score alone, measured only on synthetic composites, cannot by itself confirm that the detector has actually generalized to real photographs rather than to the particular visual style of this project’s synthetic backgrounds.

Chapter 4

Identifying Each Sign (Classification)

4.1 The task

Once Chapter 3 has produced a bounding box for a glyph, the pipeline crops out exactly that rectangle from the original photograph and hands the crop to a separate model whose only job is to answer one question: of the 171 Gardiner sign codes this project knows about, which one is this? This is the image classification task defined in Section 3.3, and it is answered by a convolutional neural network (Section 2.4) called **ResNet18**.

4.2 The dataset

The classifier is trained on 4,031 single-glyph images spanning 171 Gardiner sign classes, sourced from a published dataset matching the one used in Franken and van Gemert’s 2013 paper on automatic hieroglyph recognition, itself photographed from the interior of the Pyramid of Unas. Every image is a grayscale crop, 50 pixels wide by 75 pixels tall, showing exactly one glyph with a small margin of background around it.

This dataset is not evenly distributed across its 171 classes. The most common sign class has 448 images; the median class has only 5. Roughly 51 of the 171 classes have fewer than 3 images each, too few to reliably hold any back for measuring accuracy on that specific class, so those classes are still included in training but their real-world accuracy is simply unverified. Figure 4.1 shows the size of this gap in scale. This kind of skew, where a handful of common classes have abundant examples and most classes have only a few, is typical of a real archival dataset rather than a dataset purpose-built for machine learning, and it is addressed at training time, not by discarding the rare classes (discussed in Section 4.4).

Before training, every class’s images are split independently into training, validation, and test portions (15% held out for validation, 15% for test, the remainder for training), with a guaranteed minimum of one image in validation and one in test for any class large enough to spare them; classes with fewer than 3 images go entirely into training, since holding back even a single image from a 2-image class would leave nothing meaningful to train on for that sign.

4.3 Transfer learning: reusing a pretrained model

Training a CNN as deep as ResNet18 (18 layers) entirely from scratch normally needs a very large labeled dataset, typically hundreds of thousands of images, to learn good general-purpose filters (the edge-and-corner detectors described in Section 2.4) before it can even start learning

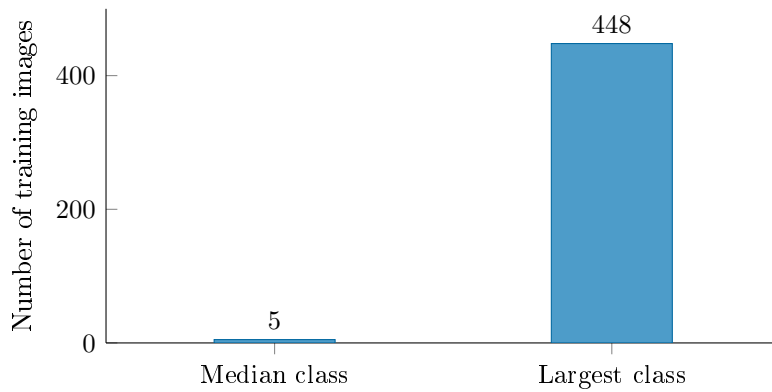


Figure 4.1: Class size imbalance in the 4,031-image training dataset. The median of the 171 sign classes has only 5 training images; the single most common class has 448, nearly two orders of magnitude more. Separately, 51 of the 171 classes have fewer than 3 images each and therefore have no held-out test data at all (Section 4.2).

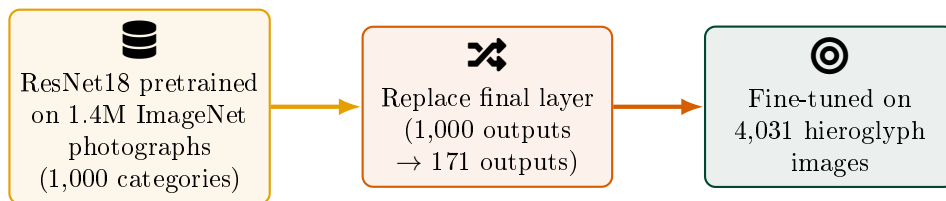


Figure 4.2: Transfer learning for the sign classifier. A ResNet18 already trained on 1.4 million general-purpose ImageNet photographs (orange) has its final, category-specific layer swapped out for a new one sized for 171 Gardiner sign classes (vermillion), then the whole network is fine-tuned on this project’s much smaller hieroglyph dataset (green). Every early layer’s ImageNet-learned weights are reused as a starting point, not discarded.

the specific task at hand. With only 4,031 training images spread across 171 classes, training ResNet18 from scratch on this dataset alone would not work well.

Transfer learning avoids this by starting from a version of ResNet18 that has already been trained on ImageNet, a very large, widely used dataset of 1.4 million photographs across 1,000 everyday object categories (dogs, cars, chairs, and so on, none of them hieroglyphs). The intuition: the early and middle convolutional layers of a network trained on ImageNet have already learned to detect edges, curves, textures, and simple shapes, which are useful visual building blocks for recognizing *any* kind of image, hieroglyphs included, even though the network has never seen one. Only the network’s final layer, the one that turns those general-purpose features into a specific class score, is specific to the 1,000 ImageNet categories and needs to be replaced.

Concretely, this project’s classifier is built by taking a ResNet18 already trained on ImageNet, discarding its final layer (which produced one score per ImageNet category), and attaching a new, untrained final layer sized for 171 Gardiner sign classes instead. Every earlier layer keeps its ImageNet-learned weights as a starting point; the new final layer starts from random values and has everything left to learn. Training then proceeds as described in Section 2.3, adjusting every weight in the network, including the reused ones, though typically by much smaller amounts than the brand-new final layer, since the reused weights are already close to useful rather than starting from nothing. Figure 4.2 shows this end to end.

4.4 Training details

Two choices made when training this classifier are worth calling out explicitly, since both are direct responses to specific properties of hieroglyphic signs rather than generic defaults.

Class-weighted loss, to counter the imbalance shown in Figure 4.1. Without correction, a model can achieve a deceptively good-looking overall accuracy simply by becoming very good at the handful of large classes (up to 448 images) while performing poorly on the many small classes (as few as 1 or 2 images), since large classes dominate the total count of training examples. Training instead weights each class’s contribution to the training signal inversely to how many images it has, so that a mistake on a rare class counts as much as a mistake on a common one, rather than being statistically drowned out.

No horizontal flipping, unlike most image classifiers. A very common, normally harmless way to artificially expand a small image dataset is **data augmentation**: applying small random transformations (rotating slightly, adjusting brightness, or mirroring left-right) to each training image so the model sees more visual variety than the raw dataset alone provides. This project applies small random rotations (up to 5 degrees, since a real photograph is rarely taken perfectly level) and small random brightness and contrast adjustments (simulating uneven lighting on a stone surface), but deliberately does *not* apply horizontal mirroring, which is standard practice for most photograph classifiers. The reason is specific to hieroglyphs: many signs are meaningfully directional (an animal or human figure facing left carries different reading-order information, and in some cases a genuinely different meaning, from the same figure facing right), so mirroring a training image can silently turn it into a misleading example of a visually similar but different sign, rather than a harmless variation of the same sign.

4.5 Results

The trained classifier reaches 97.3% accuracy on its held-out test set (620 test images, drawn from the roughly 120 of 171 classes large enough to have any test images at all, per the split described in Section 4.2). Where the model does make mistakes, those mistakes are not scattered randomly across all 171 classes; they concentrate in small clusters of signs that are visually similar to each other, for example the group of signs **G1**, **G17**, **G35**, and **G4** (a cluster of bird hieroglyphs sharing a broadly similar silhouette), and separately **M1**, **M29**, and **M17** (plant-sign glyphs with similar overall shapes). This pattern, where confusions cluster among genuinely similar-looking signs rather than occurring arbitrarily, is a form of qualitative sanity check on the trained model: a model making arbitrary, random mistakes would suggest something had gone wrong in training, whereas a model whose few mistakes concentrate on the pairs a human would also find visually similar is behaving the way a reasonably well-trained classifier should.

Chapter 5

Putting It Together: Ordering and Looking Up Signs

Chapters 3 and 4 together turn a photograph into an unordered collection of (bounding box, Gardiner code) pairs, one per glyph found. This chapter covers the two remaining steps that turn that unordered collection into the final, readable output: putting the signs into a sensible reading order, and looking up each sign’s transliteration and English gloss. Figure 5.2 shows how every stage covered so far in this report fits together into the complete system.

5.1 Reading order

A photograph’s glyphs are detected in no particular order (both the classical segmenter of Section 3.2 and the trained detector of Section 3.4 simply report whatever boxes they find, in whatever order their internal processing happens to produce them). Before the signs can be read as a list, they need to be sorted into the order a human reader would actually read them in.

This project uses a deliberately simple rule: group the detected boxes into horizontal rows by vertical position, then sort left to right within each row, reading row by row from the top of the photograph to the bottom, the same order English text is read in. Two boxes are placed in the same row if their vertical centers are close together, specifically within a distance proportional to the average height of the boxes already in that row (a fraction of box height is used rather than a fixed pixel distance, since glyphs in the same photograph can vary considerably in size, and a fixed pixel threshold tuned for large glyphs would incorrectly merge separate rows of much smaller ones). Figure 5.1 works through a concrete example.

This rule is stated plainly as an approximation, not a solved problem. Real Egyptian hieroglyphic inscriptions do not always read top-to-bottom, left-to-right; the true reading order can run in columns, or right-to-left, depending on which way the signs, especially animal and human figures, are drawn facing. Correctly following that convention would require the classifier to also determine each sign’s facing direction, which this project’s classifier does not currently do. This is documented as a known limitation, not silently glossed over (Chapter 7).

5.2 Looking up each sign’s meaning

Once the signs are ordered, each one’s Gardiner code is looked up in a table (a spreadsheet-style file, `gardiner_signs.csv`, hand-compiled from Sir Alan Gardiner’s 1957 published sign list and cross-checked against this project’s exact 171 trained classes) to retrieve its transliteration and

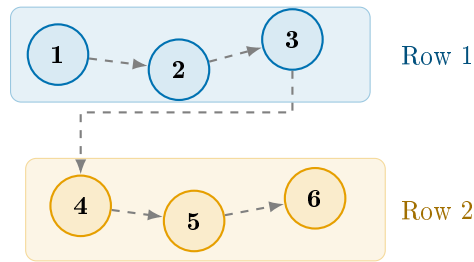


Figure 5.1: Reading-order sorting. Six detected glyphs (numbered circles, positioned where their bounding boxes were found) are first grouped into rows by vertical proximity (blue band, orange band), then read left to right within each row, row by row from top to bottom (dashed path). The resulting order, 1 through 6, is the order glyphs 1-6’s Gardiner codes are concatenated into the final output.

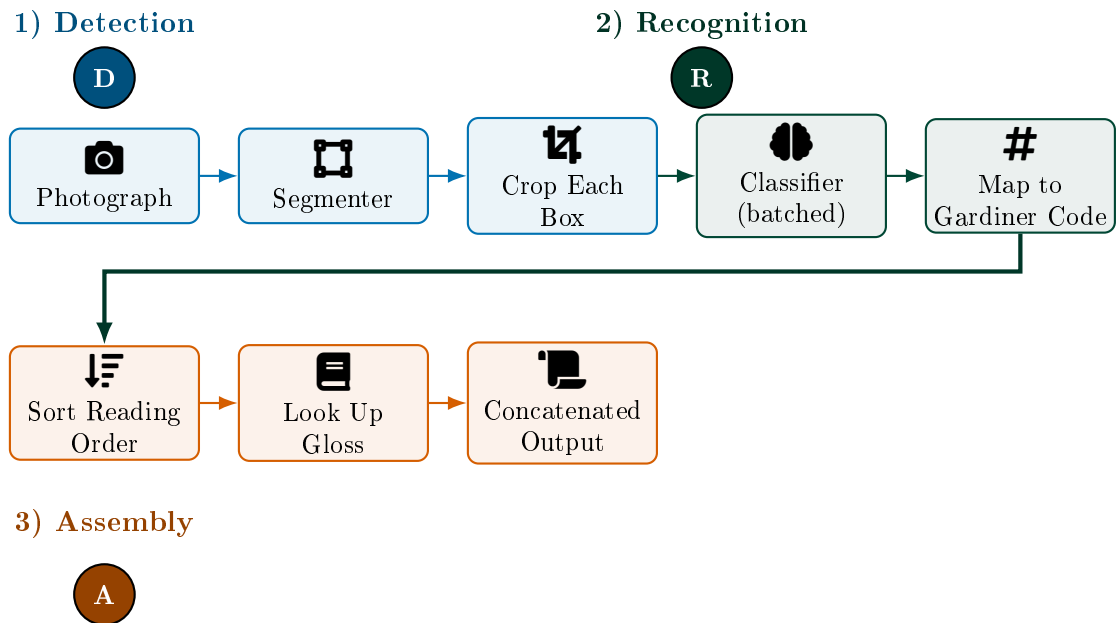


Figure 5.2: The complete pipeline, from photograph to final output, in three phases: detection (blue, Chapter 3), recognition (green, Chapter 4), and assembly (vermillion, Section 5.1 and Section 5.2). Classification runs as one batched forward pass over every cropped box from a photograph rather than once per box, which is substantially faster on both CPU and GPU than looping over boxes one at a time.

English gloss. This table is honest about its own gaps rather than inventing plausible-looking answers to fill them: a transliteration is left genuinely blank for signs called **determinatives**, which convey meaning but were never assigned a sound value in the first place, and five sign codes that either were not present in Gardiner’s original 1957 list at all, or needed a second source to resolve, are handled explicitly, four of them left with an unresolved, empty gloss and a note rather than a guessed meaning, and one (011) resolved by cross-referencing the Unicode 17.0 standard’s own hieroglyph block. If the classifier ever outputs a code that has no entry in this table at all, which would indicate a genuine mismatch between the classifier’s class list and the lookup table rather than an expected gap, the lookup deliberately raises a loud error instead of silently returning an empty result, so that kind of bug cannot pass unnoticed during development.

5.3 The complete pipeline

Figure 5.2 shows every stage covered in this report wired together. One detail worth stating explicitly, since it is a real performance decision rather than an incidental implementation choice: the classifier is run once per photograph as a single **batched** forward pass over every cropped box found in that photograph, rather than as a separate model call for each individual box. Modern deep learning hardware, whether a CPU or a GPU, is substantially more efficient processing many inputs of the same shape together in one call than processing the same inputs one at a time in a loop of many smaller calls, so batching is a meaningful speed advantage on a photograph containing many glyphs, at no cost to the result.

Chapter 6

The Demo Application

The pipeline described in Chapters 3 through 5 is exposed through a small web application built with Streamlit, a Python framework for turning a script into an interactive browser-based tool without writing separate frontend code. A visitor uploads a photograph; the application runs the full pipeline from Figure 5.2 on it and displays the result as an annotated copy of the photograph (every detected glyph outlined and numbered in reading order) alongside a card for each sign showing its Gardiner code, transliteration, gloss, and the classifier’s confidence for that particular prediction, color-coded green, yellow, or red depending on whether that confidence is high, medium, or low.

The application is written to reflect this report’s own honesty about the system’s current limitations rather than presenting the pipeline as more capable than it is: its sidebar states plainly, in the same terms as Chapter 7 of this report, that the output is not a sentence translation, that the currently active segmenter is the classical, non-learned one (Section 3.2) unless a trained detector checkpoint (Section 3.4) has actually been placed on disk, and that reading order is a simplified approximation (Section 5.1). The application checks, each time it loads, whether a fine-tuned detector checkpoint file is present, and automatically uses it in place of the classical segmenter if so, with no code change required; this is the same **Segmenter** interface substitution described in Section 3.4, made concrete as a real runtime decision rather than a design-time one.

Chapter 7

Known Limitations and Future Work

This chapter states, without softening, what this system does not currently do well. Every limitation listed here is stated elsewhere in this report at the point it first becomes relevant; this chapter collects them in one place for a reader who wants the honest summary first.

- (1) **This is not a sentence translator.** The output is a list of individual sign glosses in reading order, not fluent English prose. Producing an actual sentence translation would require modeling Middle Egyptian grammar, which this project does not attempt, and no large aligned hieroglyph-to-English sentence corpus exists to train such a system on even if it were attempted.
- (2) **The classical segmenter (Section 3.2) struggles on difficult photographs.** It performs well on clean, evenly-lit photographs but can fail on heavily weathered surfaces, uneven lighting, or cluttered scenes, for the reasons explained in Section 3.2. The trained detector described in Section 3.4 is intended to address this, but at the time of writing has not yet completed training, so its real-world performance is not yet measured.
- (3) **Reading order is an approximation.** The row-then-left-to-right rule described in Section 5.1 does not account for sign-facing direction, which genuinely determines reading order in real hieroglyphic inscriptions.
- (4) **Roughly 51 of 171 sign classes have no verified test accuracy.** These classes have too few images (fewer than 3) to hold any back for testing (Section 4.2); the classifier is still trained on them, but how accurately it recognizes those specific signs in practice is unknown.
- (5) **The trained detector’s real-world accuracy is not yet known.** As of this report, the fine-tuning run described in Section 3.4.4 has not yet been carried out; both the quantitative and qualitative evaluations described in Section 3.4.6 are still pending.

Two directions for future work follow directly from these limitations. The most immediate is completing and evaluating the trained detector’s training run, and, if its real-photograph performance genuinely improves on the classical segmenter’s, making it the default rather than an opt-in alternative. The second, larger direction, noted here as an open problem rather than a committed plan, is sign-facing detection: teaching the classifier or a separate small model to additionally report which direction a sign faces, which would allow the reading-order stage (Section 5.1) to follow the actual Egyptological convention rather than the current simplified row-major approximation.

Chapter 8

Summary

This report has walked through a complete photograph-to-gloss pipeline for Egyptian hieroglyphs, defining every machine learning concept involved from first principles along the way. The system finds glyphs in a photograph using either a fixed, non-learned image-processing procedure or a fine-tuned neural object detector; identifies each found glyph using a convolutional neural network built by adapting a model pretrained on a large, unrelated image dataset rather than trained from nothing; and assembles the per-glyph results into an ordered, looked-up output using explicit, clearly-stated rules rather than another learned model. Two specific engineering issues were documented in detail as worked examples rather than smoothed over: the risk of data leakage in a synthetically generated training set, and a real label-format mismatch that silently produced plausible-looking but unusable training data until the full pipeline was actually run end to end. The system’s classifier is measured and performing well (97.3% test accuracy); its newer trained detector is built and ready to train, but not yet evaluated; and its remaining known weaknesses, sentence-level translation, reading-order accuracy on directional inscriptions, and rare-class coverage, are stated plainly rather than implied to be solved.